Prepared by: Alvin Delos Santos Paano
Date: October 14, 2026
Subject: Technical Assessment: Integrated-Resort Data Platform ERD Review

---

Executive Summary

This technical assessment provides a comprehensive architecture review of the Integrated-Resort Data Platform (v0.1) draft ERD. While the initial draft successfully captures foundational business requirements, the current physical implementation contains several structural anti-patterns—such as premature business logic transformations, mixed fact grains, and coupled dashboard logic—that pose risks to data trust, pipeline resiliency, and BI scalability.
The primary goal of this review is to establish a scalable, production-ready, dbt-native Medallion Architecture. By enforcing strict layer separation, implementing Slowly Changing Dimensions (SCD Type 2), decoupling fact grains, and adopting standardized dbt naming and modeling conventions, this refined architecture ensures robust metric governance, flexible analytics, and an audit-ready data foundation.

ERD Review: Integrated-Resort Data Platform (v0.1)

Here is a review of the v0.1 draft ERD. The junior engineer has captured the core business requirements, but the physical implementation contains several structural anti-patterns that will cause issues with data trust, pipeline maintenance, and BI tool integration.

1. Medallion Architecture & Layer Placement

Current State Assessment

The draft applies business transformations too early and aggregates too soon.

- `bronze.cms_ratings` computes `theo_amount` and `is_active_flag` on load.
- `silver.player_monthly_theo` applies time-based aggregation in the foundation layer.

Architectural Recommendation

- **Bronze (Raw):** This layer must be an exact, append-only replica of the source systems. No business logic, no computed flags, and no metric calculations. If the ingestion pipeline breaks because a source schema changes, you want the failure isolated here.

Bronze tables should strictly contain raw columns plus metadata (e.g., `batch_id`, `_airflow_run_id` or `_dbt_loaded_at`).
    - **Data Lineage and Auditing:** Bronze must act as an exact, immutable replica of the source systems. If a stakeholder disputes a number on the executive dashboard, you need a pristine, unaltered record to check against the source system. If data is transformed as it lands, you lose that baseline audit trail.
    - **Pipeline Resiliency:** The ELT paradigm separates ingestion from transformation. If the CMS vendor changes a data type or adds a column, a dumb, append-only raw layer will gracefully accept the new data. If the Bronze layer contains load-time calculations, unexpected source changes will break the ingestion pipeline, stalling the entire platform.
    - **Historical Replayability:** If the business decides to change how `theo_amount` is calculated next year, having raw, untouched data in Bronze allows you to completely rebuild the Silver and Gold layers with the new logic. If `theo_amount` is hardcoded at the moment of ingestion, historical recalculation becomes an engineering nightmare.
- **Silver (Foundation):** Move the `theo_amount` calculation here. Silver is where you clean, cast, and conform data into your core dimensional models. Remove `silver.player_monthly_theo` entirely—aggregations belong in Gold or the BI layer.

2. Dimensional Modeling & Identity Resolution

Current State Assessment

Truncating and reloading the player dimension destroys historical context, making it impossible to answer questions like, "What was this player's tier when they generated this specific gaming rating six months ago?" Additionally, keeping CMS and Loyalty identities separate will fracture reporting.

Architectural Recommendation

- **Handling Tier Changes (SCD Type 2):** Implement a Slowly Changing Dimension Type 2 for `dim_player`. In a dbt ecosystem, use **dbt snapshots**. This will automatically track historical changes by adding `valid_from`, `valid_to`, and `is_current` flags. When joining `fact_play` to `dim_player`, join on the player ID *and* ensure the rating timestamp falls between the dimension's valid dates.
- **Identity Resolution:** Introduce a staging process that generates a conformed enterprise key (e.g., `sk_player_id` or `guest_id`). The Silver `dim_player` becomes the "Golden Record" containing both source IDs as natural keys, but downstream facts join on the conformed `sk_player_id`.

3. Fact Table Grain Strategy

Current State Assessment

Mixing individual rating events and daily summaries in a single `fact_play` table using a `record_type` flag is a severe anti-pattern. It creates "fan-out" traps and double-counting risks in BI tools if an analyst forgets to filter on `record_type`.

Architectural Recommendation

Strictly separate the grains into distinct fact tables:

1. **`fact_rating` (Transactional Grain):** One row per gaming session/rating. This captures the lowest level of detail.
2. **`fact_player_daily_summary` (Periodic Snapshot Grain):** One row per player per day. This can be built downstream from `fact_rating` to optimize dashboard performance.

4. Gold Layer & BI Integration

Current State Assessment

`gold.exec_daily_dashboard` is tightly coupled to a specific visual layout. Creating "one wide row per day" tables creates a brittle architecture.

Architectural Recommendation

The Gold layer should provide highly conformed, business-aligned data marts (Star Schemas or One-Big-Tables/OBTs) designed for self-service, not hardcoded dashboard layouts.

- Provide a well-structured `mart_gaming_daily` OBT.
- Let the BI tool handle time-intelligence functions (MTD, YTD calculations) and the final pivot into a single-row visual layout.
- **Defensive Testing:** To make the Gold layer bulletproof against upstream anomalies, implement dbt test (e.g., `uniqueness`, `not_null`, and `accepted_values`) strictly on the Silver models. If an upstream issue corrupts the Silver data, the pipeline should fail before it builds the Gold marts, ensuring executives never see a broken dashboard.

5. Metric Governance

Current State Assessment

Computing `theo` on load in Bronze and then recomputing it with a new formula in `gold.campaign_performance` guarantees mismatched numbers across reports.

Architectural Recommendation

Adopt a strict "Define Once, Use Everywhere" policy. Compute the base components (`coin_in` and `house_edge`) in the staging layer, calculate the absolute `theo_amount` per transaction in the Silver `fact_rating`, and **only** apply `SUM(theo_amount)` downstream.

6. Proposed Target State Model (DBML)

Note: This revised ERD implements the naming conventions suggested in Section 8, dropping the medallion prefixes and utilizing standard dbt structural prefixes (`fct_`, `dim_`, `mart_`).

```
// ==========================================
// SILVER LAYER (Core Dimensional Model)
// ==========================================

Table dim_player {
  sk_player_id   varchar   [pk, note: 'Surrogate/Conformed Key']
  cms_player_id  varchar   [note: 'Natural Key 1']
  member_no      varchar   [note: 'Natural Key 2']
  first_name     varchar
  last_name      varchar
  tier           varchar   [note: 'Tracked via dbt snapshot']
  valid_from     timestamp
  valid_to       timestamp
  is_current     boolean
}

Table dim_campaign {
  sk_campaign_id varchar   [pk]
  campaign_code  varchar
  campaign_name  varchar
  start_date     date
```

```
  end_date       date
}

Table fct_rating {
  sk_rating_id   varchar   [pk]
  sk_player_id   varchar
  rating_ts      timestamp
  game_type      varchar
  coin_in        decimal
  house_edge     decimal
  theo_amount    decimal   [note: 'Computed here: coin_in *
house_edge']
}

Table fct_campaign_redemption {
  sk_redemption_id varchar   [pk]
  sk_campaign_id   varchar
  sk_player_id     varchar
  redemption_ts    timestamp
  offer_amount     decimal
}

// ==========================================
// GOLD LAYER (Business Marts / OBTs)
// ==========================================

Table mart_player_daily_summary {
  date_day             date      [pk]
  sk_player_id         varchar   [pk]
  daily_coin_in        decimal
  daily_theo           decimal
  days_since_last_play int       [note: 'Used for active/inactive
status']
  is_active_30d        boolean
}

Table mart_campaign_performance {
  sk_campaign_id       varchar   [pk]
```

```
  sk_player_id         varchar   [pk]
  is_invited           boolean
  is_redeemed          boolean
  incremental_theo     decimal   [note: 'Post-campaign theo minus
baseline theo']
}

// ========================================
// RELATIONSHIPS
// ========================================

Ref: fct_rating.sk_player_id > dim_player.sk_player_id
Ref: fct_campaign_redemption.sk_player_id > dim_player.sk_player_id
Ref: fct_campaign_redemption.sk_campaign_id >
dim_campaign.sk_campaign_id

Ref: mart_player_daily_summary.sk_player_id > dim_player.sk_player_id
Ref: mart_campaign_performance.sk_player_id > dim_player.sk_player_id
Ref: mart_campaign_performance.sk_campaign_id >
dim_campaign.sk_campaign_id
```

7. Identity Mapping & DAG Modularity

Current State Assessment

Handling identity resolution directly inside the final dimension models usually makes the SQL too dense and difficult to debug.

Architectural Recommendation

Utilize an intermediate (`int_`) model for identity resolution (e.g., `int_player_identity_map`). This model should sit between your staging tables (`stg_cms_players`, `stg_loyalty_members`) and your final `dim_player`. It joins the staging tables, deduplicates records, resolves the identities, and generates the conformed enterprise key. This modularizes your DAG, makes the logic testable in isolation, and keeps the final dimension model clean.

- **Prevents Metric Drift:** Centralizing identity resolution prevents downstream metric drift (such as double-counting a guest's theoretical win). If a single person is treated as multiple distinct entities in the data models, player-level aggregations become fragmented or artificially inflated.

```
□[stg_cms_players]       \
(cms_player_id)          \
                          +--> [int_player_identity_map] --->
[dim_player]
                         /     (Generates sk_player_id)       (Final
Attributes & SCD2)
[stg_loyalty_members]   /
(member_no)
```

□8. dbt Naming Conventions

Current State Assessment

The draft mixes Medallion layer names with physical table names (e.g., `bronze.cms_ratings`) and ties Gold models directly to specific dashboards (e.g., `gold.exec_daily_dashboard`), violating dbt best practices.

Architectural Recommendation

1. **Drop Medallion Prefixes:** Manage Medallion layers via target schemas in `dbt_project.yml`, not physical file names.
2. **Standardize Structural Prefixes:** Adopt standard dbt prefixes to communicate a model's purpose in the DAG: `src_` (Sources), `stg_` (Staging), `int_` (Intermediate), `dim_` (Dimensions), `fct_` (Facts), and `mart_` (Marts).
3. **Entities over Reports:** Never name a model after a dashboard. Dashboards change constantly. Name the mart after the business entity or process (e.g., `mart_gaming_daily`) so it remains reusable.